

Performance Testing

Date	3 July 2026
Team ID	XXXXXX
Project Name	OptiCrop – AI-Based Crop Recommendation System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the Smart Lender application performs under different user loads and verifies that the Machine Learning prediction system responds efficiently and reliably.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	Crop Recommendation Form, Flask Prediction Route (/predict), Crop Recommendation
Test Environment	Local System (Windows 11)
Test Date	02-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home Page Loading	20	60	Home page should load successfully with minimal response times.
2	Crop Recommendation Form Submission	50	120	Soil and environmental data should be submitted successfully without errors.
3	Crop Recommendation Prediction	100	180	Recommended crop should be accurately with no server failures..
4	Multiple Users Accessing the Application	75	120	Application should remain responsive under concurrent user requests.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 seconds	pass	Response time within acceptable limits.
2	Response Time (Max)	< 5 seconds	3.4 seconds	Pass	Stable during peak load.
3	Throughput (Req/sec)	>40 Req/sec	52 Req/sec	Pass	Good request handling
4	Error Rate	< 1%	0.1%	Pass	Very few request failures
5	CPU Utilization	< 80%	61%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	55%	pass	Memory usage within limits.

Step 4: Observations & Analysis

Key Findings:	- The OptiCrop Flask application successfully predicts the most suitable crop based on user inputs. - The Machine Learning model (Random Forest Classifier) achieved high accuracy in crop recommendation. - The application provides fast and reliable predictions with a user-friendly interface.
Bottlenecks Identified:	- A slight increase in response time was observed when multiple users accessed the application simultaneously. - Large input processing (e.g., extreme weather data) may slightly increase prediction response time under heavy load.
Optimization Steps Taken:	- Optimized the Flask application by preprocessing user input efficiently before prediction. - Used a trained Random Forest model and StandardScaler to reduce prediction time. - Improved code organization and input validation to ensure smooth and reliable prediction.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

